

Selective Adversarial Causal Oversight

Benchmark prototype + countermodel-grounded reference verifier for selective causal refusal

Jingbei Niu • Zeyuan Tong • Yuhang Li

Zhejiang University, College of Computer Science and Technology

Exploratory snapshot | 3 seeds | 10/family | diff=0.55 | 95% CI



浙江大學
ZHEJIANG UNIVERSITY

1. Motivation & Abstract

Causal oversight systems must decide when a public causal claim is valid and when the available evidence is insufficient. Current prompt-only and heuristic judges often trade unsafe acceptance against over-refusal. We study this tension as a benchmark-first problem: the verifier receives only public scenario evidence, while gold SCMs and labels remain evaluator-only. The current artifact is an exploratory prototype, not a mature benchmark release.

Key contributions

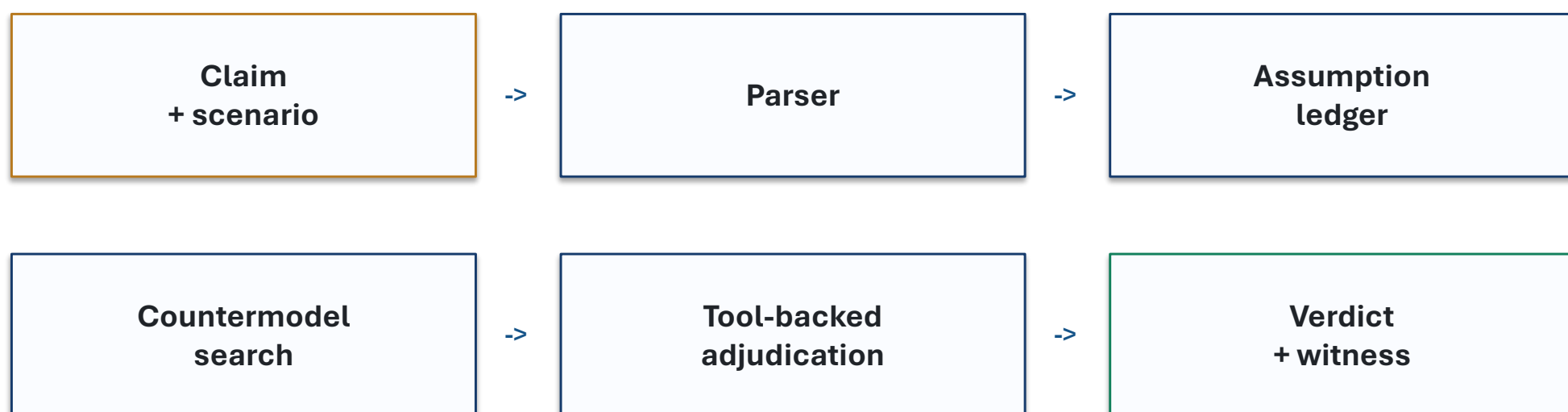
Selective verdict contract: VALID / INVALID / UNIDENTIFIABLE.
Countermodel-grounded verifier with evaluator-only gold labels.
Frozen evidence package: main tradeoff, API baselines, OOD buckets, and ablations.

Task contract

Input: public scenario + causal claim only.
Hidden: SCM, labels, and hidden variables.
Risk metric: unsafe acceptance plus wise refusal.

2. Method Overview

The verifier turns a claim into an assumption ledger, searches for public-compatible counter-SCM witnesses, and uses Pearl-style tools as supporting evidence before issuing a selective verdict.



Counter-SCM witnesses are diagnostic certificates, not leaked gold evidence.

3. Setup / Data / Tasks

Frozen experiment snapshot

Synthetic benchmark families with train/dev/test_iid/test_ood splits.
Three seeds; 10 cases per family; difficulty 0.55.
API baseline matrix: 5 models, 1245 predictions.
All poster numbers are generated from frozen JSON artifacts.

Validity boundary

Prompt-protocol-specific API baselines, not universal LLM claims.
Graph/mechanism OOD buckets are saturated smoke tests.
Release requires larger synthetic runs and real-grounded dual audit.

4. Main Technical Details

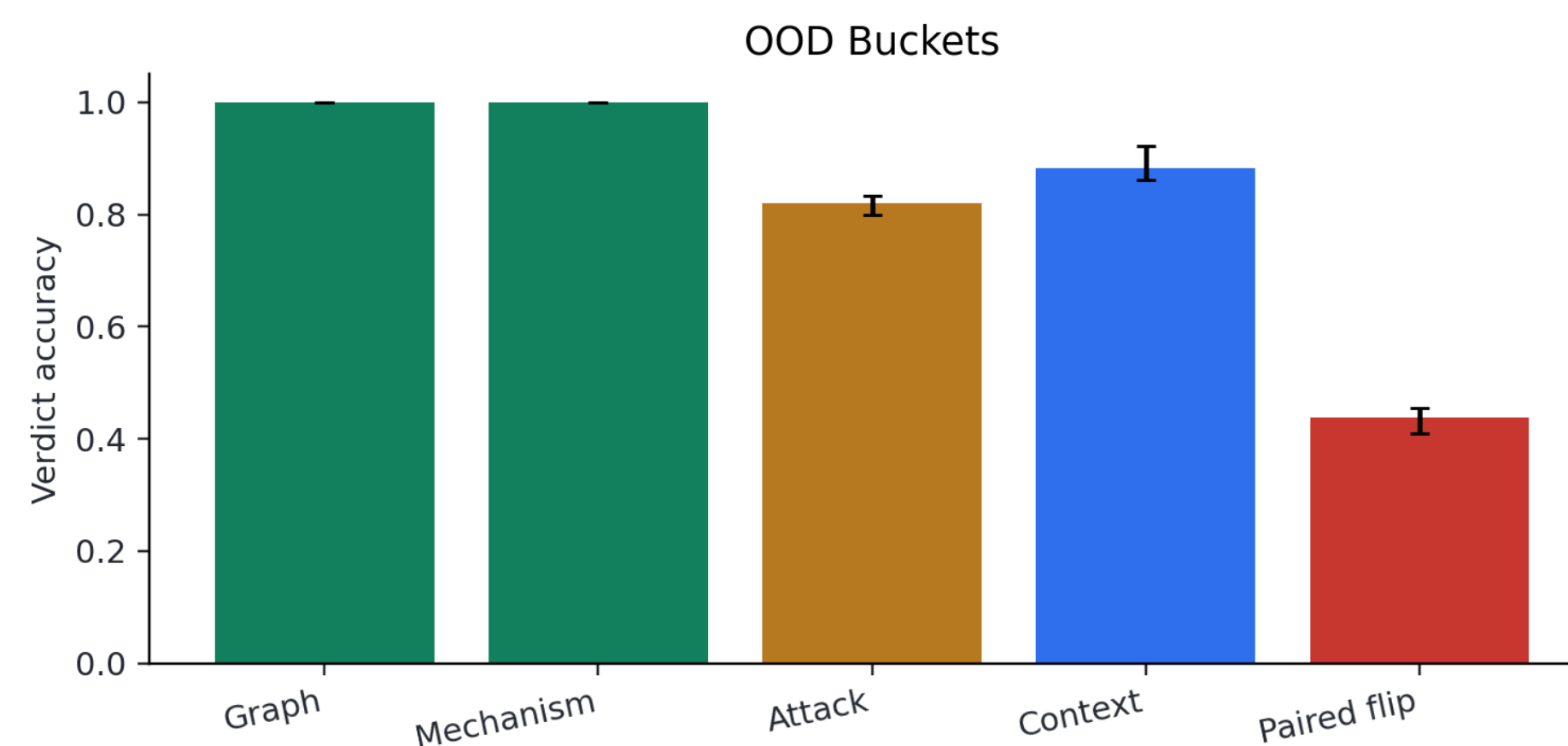
The technical object is selective causal verification: accept identifiable valid/invalid claims, but refuse when public evidence admits multiple compatible causal worlds.

Selective target: maximize OOD accuracy while minimizing unsafe acceptance and over-refusal.

Key mechanism: public-compatible countermodels witness missing identification assumptions.

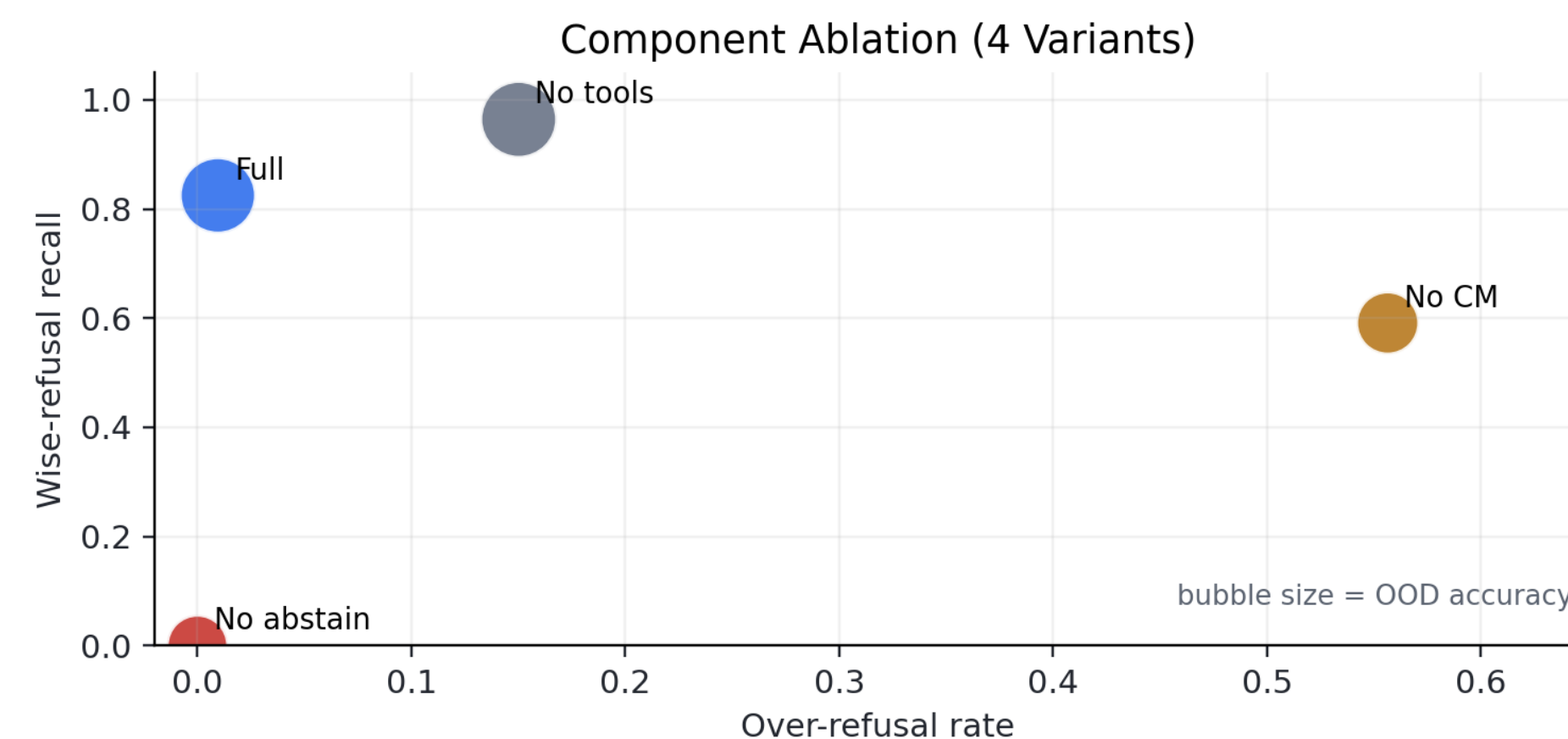
Interpretation

A countermodel witness supports UNIDENTIFIABLE rather than forcing binary validity.
Pearl-style tools constrain the verifier but do not expose evaluator-only labels.
The design targets unsafe acceptance without collapsing into blanket refusal.



OOD stress reading

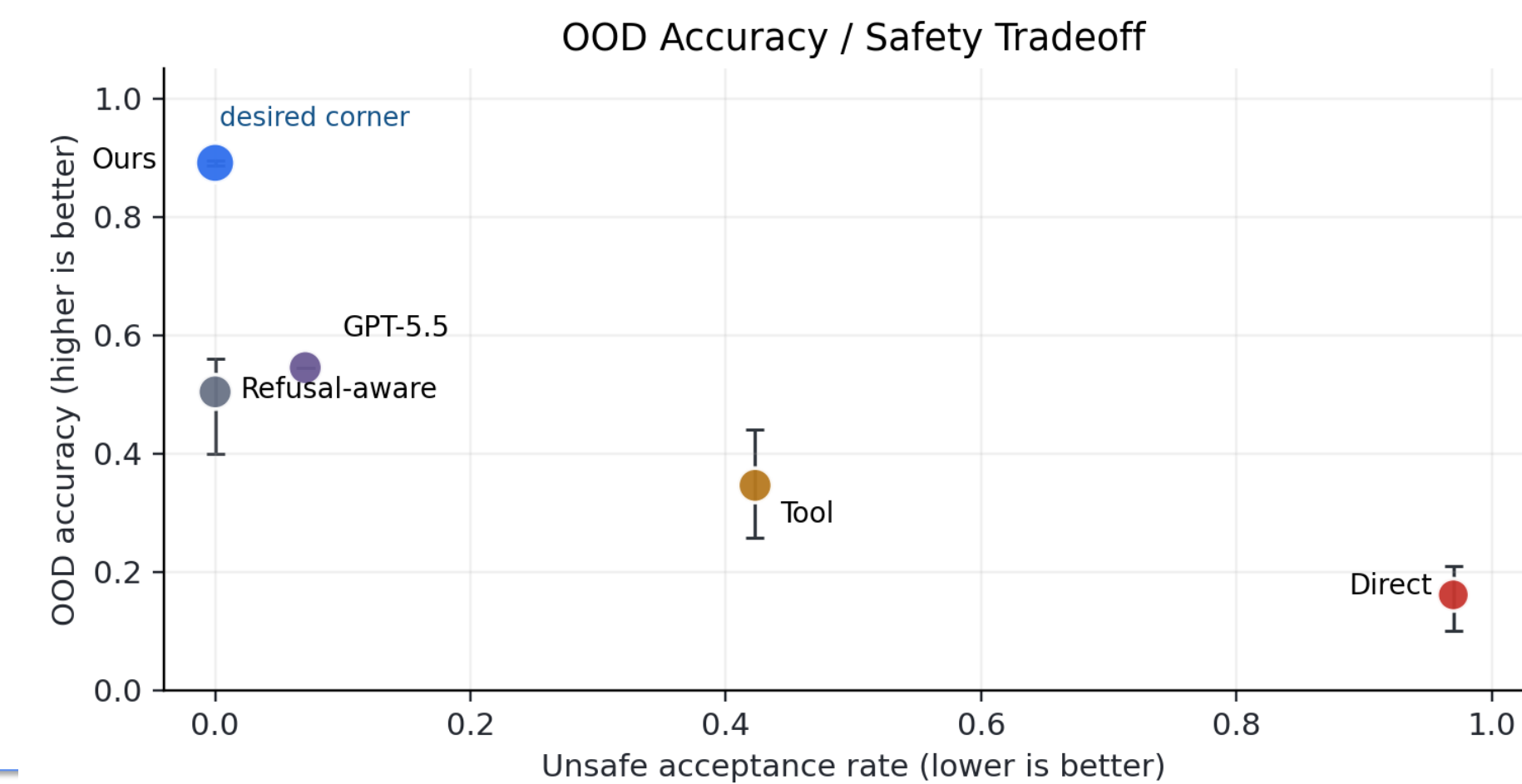
Paired-flip is the clearest current failure slice: acc 0.439.
Graph/mechanism buckets are saturated smoke tests.
Use these buckets as diagnostics, not final robustness proof.



Ablation uses four frozen component variants; bubble size encodes OOD accuracy.

5. Main Results / Examples

The strongest quantitative evidence is the selective OOD tradeoff, followed by bucket-level stress tests and component ablations.



Result caption

Ours: OOD acc 0.891, unsafe accept 0.000.
GPT-5.5 prompt baseline: OOD acc 0.545, unsafe accept 0.071.
The evidence supports a prototype diagnostic claim, not a final benchmark release.

Examples

Example A: unidentifiable claim

Claim: association between therapy_flag and yield_score is treated as causal.

Gold label: UNIDENTIFIABLE.

Reason: public evidence leaves confounding or reverse causality open.

Example B: verifier response

Output: refuse the causal conclusion.

Witness: construct a public-compatible counter-SCM.

Takeaway: refusal is evidence-sensitive, not blanket abstention.

Validity gates before release

Leakage check: oracle-positive 1.000/1.000; gain is contamination.
API baselines: 5 models, 1245 predictions; best OOD GPT-5.5 acc 0.545, unsafe 0.071.
Next gates: larger synthetic runs, real-grounded dual audit, frozen model versions.

Frozen artifact manifest: outputs/paper_facing/manifest.json | full_llm_baseline_matrix_completed